Practical Work 3: MPI File Transfer

Tran Tuan Vu - 23BI14459

December 11, 2025

1 Implementation Choice

For this practical work, we selected **OpenMPI** as our MPI implementation. We chose it because:

- It is one of the most widely used, open-source implementations of the MPI standard[cite: 641].
- It is readily available in standard Linux repositories (via `apt`) and macOS package managers.
- It provides robust support for TCP/IP networks, which simulates the distributed environment effectively even on a single machine.

2 System Design

Unlike the Client-Server model in TCP or RPC where two distinct programs are compiled, MPI uses a **Single Program, Multiple Data (SPMD)** architecture.

- We launch the same executable twice using `mpiexec -n 2`.
- We use the **Rank** (Process ID) to differentiate roles:
  - **Rank 0:** Acts as the Receiver (Server).
  - **Rank 1:** Acts as the Sender (Client).

3 Protocol Design

We designed a synchronous blocking protocol using specific **Tags** to identify the type of data being sent:

1. **Tag 0 (Filename):** Sender transmits the filename string.
2. **Tag 1 (File Size):** Sender transmits the file size as a `long` integer.
3. **Tag 2 (File Data):** Sender transmits the actual file content bytes.

4 Implementation

4.1 Main Logic

The program branches based on the rank obtained from `MPI_Comm_rank`.

```
int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 1) {
        // Rank 1 acts as Client (Sender)
        run_client(argv[1]);
    } else if (rank == 0) {
        // Rank 0 acts as Server (Receiver)
        run_server();
    }
    MPI_Finalize();
}
```

Listing 1: Role separation logic

4.2 Data Transfer

We utilize standard `MPI_Send` and `MPI_Recv` functions.

```
// Send File Size
MPI_Send(&file_size, 1, MPI_LONG, 0, TAG_FILESIZE, MPI_COMM_WORLD);

// Send File Data
fread(buffer, 1, file_size, fp);
MPI_Send(buffer, file_size, MPI_CHAR, 0, TAG_FILEDATA, MPI_COMM_WORLD);
```

Listing 2: Sender Logic

5 Experimental Results

We tested the system by transferring a text file named `testfile.txt`.

```
$ mpiexec -n 2 ./mpi_transfer testfile.txt
[Rank 1] Sending file 'testfile.txt' (28 bytes) to Rank 0...
[Rank 0] Waiting for incoming connection...
[Rank 0] Receiving 'testfile.txt' (28 bytes)...
[Rank 0] File saved successfully as 'received_testfile.txt'.
[Rank 1] Transfer complete.
```

Terminal Output

Verification:

```
$ diff testfile.txt received_testfile.txt
# (No output indicates files are identical)
```